

Задача А. Лифт

Автор задачи – В. Гуровиц, автор разбора – В. Батаев

В задаче надо найти два этажа, ближайших (снизу и сверху) к данному, N-му этажу, на котором останавливается лифт, и выбрать, на какой из них везти груз выгоднее. Для нахождения ближайшего нижнего этажа найдем целое A, такое что искомый этаж будет иметь номер $(A \cdot K + 1)$. Для этого надо разделить нацело $(N-1)$ на K. И стоимостью подъема на нужный этаж N будет разность между этажами, помноженная на 200 (стоимость подъема на 1 этаж): $200 \cdot (N - (A \cdot K + 1))$. Далее, ближайший сверху этаж, на котором останавливается лифт, будет иметь номер $(A+1) \cdot K + 1$. Единственная проблема возникает, если он не существует, то есть $(A+1) \cdot K + 1 > M$ (в этом случае ответ уже получен). Если же этот этаж существует, то стоимость спуска будет равна $100 \cdot ((A+1) \cdot K + 1 - N)$. Выбрав минимум из двух стоимостей, получим искомый ответ.

Текст программы (Delphi 6.0):

```
{ $Apptype Console }

uses SysUtils;

var
  m, n, k: longint;
  a, res: longint;
begin
  assign(input, 'a.in');
  reset(input);
  read(m, n, k);
  close(input);
  a := (n-1) div k;
  res := 200 * (n - (a * k + 1));
  if ((a+1) * k + 1 <= m) and (res > 100 * ((a+1) * k + 1 - n)) then
    res := 100 * ((a+1) * k + 1 - n);
  assign(output, 'a.out');
  rewrite(output);
  writeln(res);
  close(output);
end.
```

Задача В. Маршрутное такси

Автор задачи – В. Гуровиц, Автор разбора – В. Антонов

Это одна из самых простых задач на олимпиаде.

Понятно, что для существования решения требуется, чтобы общее количество пассажиров было кратно трём.

Теперь, когда уже известно существование решения, найдём наименьшее количество пассажиров, которое придется пересадить. Пусть $Cnt1$, $Cnt2$, $Cnt3$ – количества пассажиров в первой, второй и третьей маршрутках соответственно, Cnt – среднее арифметическое $Cnt1$, $Cnt2$, $Cnt3$, а $S = (|Cnt1 - Cnt| + |Cnt2 - Cnt| + |Cnt3 - Cnt|)$. Несложно понять, что S – количество пассажиров, которых нужно пересадить, умноженное на два, т.е. ответом будет $S \div 2$.

Задача С. Электропоезд

Автор задачи – В. Матюхин, автор разбора – В. Антонов

Это несложная техническая задача. В ней требовалось аккуратно реализовать считывание и вывод времени в формате ЧЧ:ММ, получение времени прибытия на станцию по времени прибытия на предыдущую и количеству минут, затраченных на переезд между станциями. При данных в задаче ограничениях вторую из операций можно было реализовать двумя способами:

- 1) реализовать прибавление ко времени одной минуты и получать нужное время путём прибавления по минуте в цикле;
- 2) реализовать прибавление произвольного количества минут, удовлетворяющих условию задачи, ко времени.

Остаётся не забыть про переход через 24 часа.

Теперь рассмотрим, как можно было это всё реализовать на Delphi. Положим, что в переменной *H* хранятся текущие часы, а в переменной *M* – текущие минуты.

Для считывания времени понадобятся две дополнительные переменные: *S* типа String[2] и *Tmp* типа Char.

```
Read(S, Tmp); {Считываем первые два символа – часы, а затем двоеточие}  
H:=StrToInt(S);  
Read(S); {Считываем минуты}  
M:=StrToInt(S);
```

Теперь рассмотрим прибавление *DM* минут к времени, хранящемуся в переменных *H* и *M*:

```
H:=(H + (DM+M) Div 60) Mod 24;  
M:=(DM+M) Mod 60;
```

Для вывода времени в формате ЧЧ:ММ понадобится два условных оператора:

```
If H<10 Then Write('0'); {Выводим часы и двоеточие}  
Write(H, ':');  
  
If M<10 Then Write('0'); {Выводим минуты}  
WriteLn(M);
```

Задача D. Такси

Автор задачи и разбора – А. Тимофеев

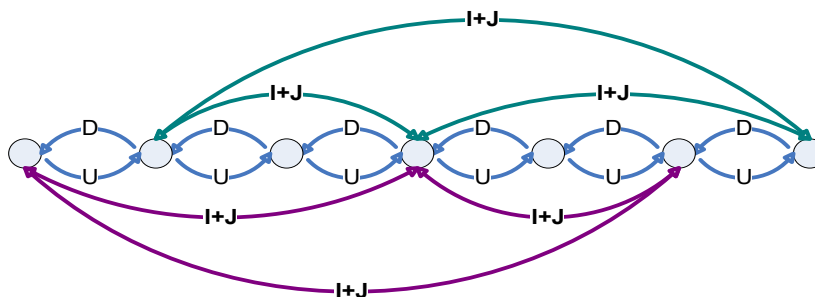
"Тише едешь – дальше будешь."

Заметим, что минимум затрат на такси достигается при такой рассадке сотрудников по такси, что i -й по величине длины поездки сотрудник (1-й тот, чей путь до дома максимален) сидит в i -м по стоимости такси (1-е такси – такси, тариф в котором минимален). Таким образом, для того, чтобы вывести номер такси, в котором при такой рассадке окажется i -й сотрудник, требуется отсортировать расстояния и тарифы в порядке убывания и возрастания соответственно.

Задача Е. Лифты

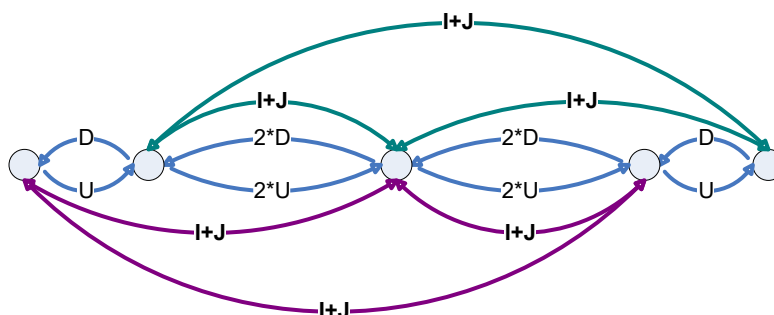
Автор задачи - В.Гуровиц, автор разбора - А.Шестимеров

Для нахождения маршрута минимальной стоимости сначала построим по условию задачи граф. Тогда для нахождения решения можно воспользоваться одним из стандартных алгоритмов поиска кратчайшего пути во взвешенном ориентированном графе (см., например, "Алгоритмы: построение и анализ", Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн). Простейший (но не оптимальный) способ представления небоскрёба в виде графа такой: пусть каждый этаж соответствует вершине, а лифты и лестницы – ребрам, веса которых – стоимости перемещения груза по данному пути (см. рис.).



Получается R вершин и $2 \cdot R + \sum_{i=1}^L (K_i - 1)^2$ ребер, где R – число этажей. При заданных в задаче ограничениях (10^6 вершин, 10^6 ребер) ни один из алгоритмов не способен найти кратчайший путь в таком графе.

Попробуем уменьшить число вершин – заметим, что этажи, на которых не останавливается ни один лифт можно удалить:

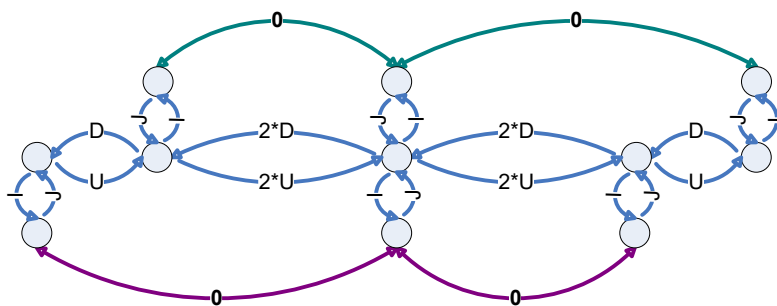


Тогда число вершин порядка $\sum_{i=1}^L K_i$, а число ребер так же порядка $\sum_{i=1}^L (K_i - 1)^2$.

Получаем максимум 10^3 вершин и 10^6 ребер, и соответственно, можно применить алгоритм Дейкстры.

Если ещё немного преобразовать граф, то можно уменьшить число ребер¹:

¹ Вершины, связанные нулевыми ребрами можно слить в одну



Тогда число ребер пропорционально числу вершин, и с помощью алгоритма Дейкстры с использованием кучи можно решить задачу со значительно большими ограничениями.

Задача F. Саморасшифровавшаяся шифровка

Автор задачи – Е.Андреева, автор разбора – А.Гусаков

Будем считать, что Петя записывал не буквы, а числа от одного до N^2 , и что полученный им шифр совпадает с последовательностью $1, 2, \dots, N^2$. Пусть после применения шифровки к полученному тексту получается последовательность $f(1), f(2), \dots, f(N^2)$, причем $f(i) < f(j)$ при $i < j$. Последовательность $f(1), f(2), \dots, f(N^2)$ будем называть *перестановкой* чисел от 1 до N^2 . Рассмотрим ориентированный граф на вершинах, занумерованных числами от 1 до N^2 , в котором есть ребро из i в j тогда и только тогда, когда $f(i)=j$ (f переводит i в j). Ясно, что в каждую вершину входит и выходит ровно по одному ребру. Следовательно, наш граф разбивается на независимые циклы (докажите это). Обозначим длины этих циклов $k_1, k_2, \dots, k_s$. Предположим теперь, что Петя применил к исходной последовательности перестановку f ровно X раз и вновь получил исходную последовательность. Это означает, например, что элементы, принадлежащие первому циклу “прошли” по нему целое число раз. Значит, X делится на любое k_i . Поскольку нам нужно наименьшее такое X , то $X = \text{НОК}(k_1, k_2, \dots, k_s)$. Обратно, если взять такое X , то после применения перестановки f ровно X раз к исходной шифровке, получим ее же.

Таким образом, решение задачи разбивается на 2 этапа:

- 1) построение перестановки и вычисление длин ее циклов;
- 2) вычисление НОК.

Опишем вкратце реализацию этих этапов. Чтобы получить перестановку f нужно просто, следуя описанному в условии способу, зашифровать последовательность чисел $1, 2, \dots, N^2$. Получится последовательность чисел $f(1), f(2), \dots, f(N^2)$. Чтобы получить последовательность $k_1, k_2, \dots, k_s$, нужно пройти по всем циклам описанного графа.

Осталось вычислить НОК. Для этого можно разложить на простые множители все k_i . Далее для каждого простого числа p_j выберем наибольшую степень m_j , в которой оно входит в разложения чисел $k_1, k_2, \dots, k_s$. После этого нужно просто перемножить числа $p_j^{m_j}$. Получится НОК, которое и будет ответом в задаче. Здесь есть еще одна проблема. При данных ограничениях результат может не поместиться в стандартных типах данных. Поэтому придется использовать длинную арифметику, то есть хранить результат в виде массива цифр и домножать его точно так же, как это делается в столбик.

Заметим, что в этой задаче сложность нахождения НОК с использованием предложенного алгоритма составляет порядка $n^2 \sqrt{n^2} = n^3$ (чтобы разложить k_i на простые множители надо перебирать все его делители от 2 до $\sqrt{n^2}$). При данных в задаче ограничениях такой алгоритм работает достаточно быстро. Отметим, однако, еще один способ нахождения НОК, который работает быстрее.

На i -м шаге алгоритма будем получать $n_i = \text{НОК}(k_1, k_2, \dots, k_i)$. Пусть n_{i-1} уже найдено. Тогда $n_i = (n_{i-1} * k_i) / \text{НОД}(n_{i-1}, k_i)$. Таким образом, задача сводится к нахождению НОД двух чисел. Для этого можно воспользоваться стандартным алгоритмом Евклида: $\text{НОД}(a, b) = b$, если $a = 0$ и $\text{НОД}(b, b \bmod a)$ иначе. Этот алгоритм работает логарифмическое по $a * b$ время. Но поскольку в этой задаче приходится работать с длинными числами, а деление длинного числа на длинное достаточно трудно в реализации, целесообразно использовать так называемый бинарный алгоритм Евклида, а именно:

- 1) $\text{НОД}(2*a, 2*b) = 2 * \text{НОД}(a, b)$
- 2) $\text{НОД}(2*a+1, 2*b) = \text{НОД}(2*a+1, b)$
- 3) $\text{НОД}(2*a+1, 2*b+1) = \text{НОД}((2*a+1)-(2*b+1), 2*b+1) = \text{НОД}(2*(a-b), 2*b+1)$

Случай 1) соответствует поиску НОДа двух четных чисел, 2) – четного и нечетного, 3) – случай двух нечетных, причем первое из них больше второго. Ясно, что этот алгоритм имеет оценку сложности асимптотически не хуже, чем стандартный, но здесь мы должны реализовывать лишь деление длинного числа на 2, что гораздо проще сделать.

Задача G. Наибольшая пилообразная подпоследовательность

Автор задачи – Е.Андреева, автор разбора – В.Антонов

Изменим нашу задачу на задачу о нахождении длины наибольшей пилообразной подпоследовательности, которую можно получить из данной последовательности.

Рассмотрим пилообразную последовательность длины l . Она состоит из возрастающих и убывающих подпоследовательностей, суммарное количество которых равно $l-1$.

Теперь посмотрим на заданную последовательность. Возможны два варианта:

- 1) Заданная последовательность состоит из одинаковых элементов. Тогда ответ очевиден.
- 2) В заданной последовательности есть хотя бы одна возрастающая или убывающая подпоследовательность. Удалим из неё все подряд идущие одинаковые элементы (понятно, что это нужно сделать, чтобы последовательность стала пилообразной). Получившаяся последовательность состоит из последовательно записанных возрастающих и убывающих подпоследовательностей, в которых соседние пересекаются по одному элементу. К примеру, последовательность 1 2 3 4 3 1 5 состоит из двух возрастающих (1 2 3 4 и 1 5) и одной убывающей (4 3 1). Понятно, что ответом будем количество таких подпоследовательностей (обозначим его за k), увеличенное на единицу. Почему ответ не может быть больше? Предположим, что это так. Но тогда существует как минимум $k+1$ возрастающих и убывающих подпоследовательностей, а в заданной последовательности их всего k . Значит, предположение неверно.

Задача Н. Тарифы

Автор разбора – В. Антонов

Воспользуемся методом динамического программирования.

Заметим, что ежемесячная абонентская плата для тарифа не будет совпадать ни с одной суммой, которую готовы заплатить абоненты, только в том случае, если этим тарифом никто не пользуется. Действительно, пусть это не так. Тогда рассмотрим группу абонентов, пользующуюся этим тарифом. Поднимем абонентскую плату до минимального значения суммы, которую готовы заплатить эти абоненты. При этом заработок оператора сотовой связи увеличится, и никто из абонентов не меняет тарифного плана.

Для решения задачи отсортируем клиентов по неубыванию суммы, которую они готовы тратить на связь в месяц, и запишем эти суммы в массив A . После этого проведём последовательное вычисление элементов массива $Ans[i, j]$, в элементе с индексами i, j которого будет храниться максимальная сумма, которую может заработать компания при обслуживании первых i клиентов и использовании ровно j тарифных планов.

Как же вычислить $Ans[i, j]$? Воспользуемся вышеизложенным утверждением. Для каждого $1 \leq l \leq i$ попробуем подключить абонентов $A[i-l+1], \dots, A[i]$ к тарифному плану, абонентская плата которого, совпадает с той суммой, которую может заплатить $A[i-l+1]$ абонент. Т.к. абоненты отсортированы по неубыванию суммы, которую они готовы тратить на сотовую связь, то всех перечисленных абонентов устроит этот тарифный план. Остаётся выбрать максимум по всем возможным l .

При инициализация массива Ans присвоим $Ans[i, j]$ ноль, если $i=j=0$, и минус бесконечность иначе.

Не стоит забывать, что не может быть двух тарифных планов с одинаковой абонентской оплатой.

После подсчёта $Ans[i, j]$ для всех возможных i и j , максимальная сумма, которую сможет заработать оператор, следует искать среди элементов $Ans[N, 1], \dots, Ans[N, K]$.

Ответ можно будет восстановить, если для каждого $Ans[i, j]$ запоминать l , для которого прибыль была максимальна. Если в ответе получилось меньше чем K тарифов, то для оставшихся можно вывести любые числа, удовлетворяющие условиям.

Задача I. Трехмерные ладьи

Автор задачи и разбора – А.Шестимеров

Построим три проекции расстановки ладей на грани куба. Тогда, фиксированная клетка (x, y, z) бьется хотя бы одной ладьей, если какая-то из ладей бьет хотя бы одну из клеток (x, y) , (y, z) , (x, z) на соответствующих проекциях. Таким образом, проверку одной клетки можно организовать за $O(1)$.

```
for x:=1 to n do
  for y:=1 to n do
    for z:=1 to n do
      if (xy[x,y]=0) and (zy[z,y]=0) and (xz[x,z]=0) then // .... нашли небьющуюся клетку.
```

Но такое решение работает за время N^3 и при заданных ограничениях на N не проходит по времени. Чтобы оптимизировать решение, можно использовать битовые операции, тогда за одну операцию мы проверим сразу несколько клеток (например, 32 или 64):

```
for y:=1 to n do
  for z:=1 to n do
    if yz[y,z]=0 then
      for k:=1 to n div 64 do
        if (yx[y,k] or zx[z,k]) <> 264-1 then // ... группу клеток, в которых есть небьющиеся.
```

Тогда время работы будет $N^3/64$ или для максимального N порядка 10^7 .

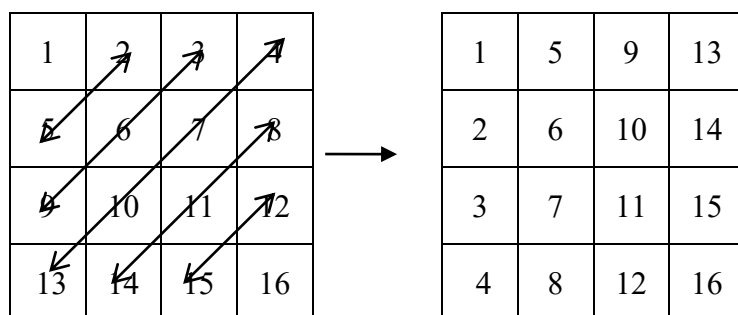
Задача J. Историки и археологи

Автор задачи – А.Лахно, автор разбора – М.Иванов

Судя по результатам олимпиады, задача J уступала по сложности лишь самой последней в списке задач: K. Действительно, задача на первый взгляд кажется неприступной. По всей видимости участники не могли заметить в ней самого главного: изюминки! А без знания этой самой изюминки решить задачу на полный бал было невозможно ☹. Кроме того, сложность представляло условие задачи: эдакая смесь русского языка с математическими формулами. Прочтите в условии еще раз, что собой представляет этап переселения

«На каждом этапе переселения переселяются сообщества внутри некоторого квадрата по следующему правилу. Если переселение происходит внутри квадрата, левой верхней клеткой которого является клетка (x_1, y_1) , а правой нижней - (x_2, y_2) , то сообщество, проживавшее в поселении с координатами (x, y) ($x_1 \leq x \leq x_2, y_1 \leq y \leq y_2$) переселяется в поселение с координатами $(x_2 - (y_2 - y), y_2 - (x_2 - x))$ »

Под этим понималось просто симметрия квадрата относительно диагонали, направленной вниз-направо (на рисунке это диагональ, соединяющая клетки 1 и 16), если вы этого не поняли – обязательно разберитесь с формулами!



Таким образом, надо было определить, можно ли из матрицы получить другую с помощью операции ротации квадратов. Никакой перебор тут конечно не поможет: в матрице 30×30 всего $30 * 30 = 900$ квадратов 1×1 , $29 * 29 = 841$ квадрат 2×2 , $28 * 28 = 784$ квадрата 3×3 и так далее, всего - 9455 квадратов! То есть в худшем случае (матрицы порядка 30×30) каждый ход можно сделать 9455 способами, то есть перебор сработает за 9455^N , где N – количество этапов переселения, за которое из одной матрицы можно получить другую.

Теперь приступим к главному: изюминке задачи. Заметим, что при ротации квадрата набор чисел (идентификаторы народов), находящиеся на каждой из диагоналей, направленных вверх-направо сохраняется (на рисунке эти диагонали обозначены стрелками). Таким образом, необходимым условием возможности получения из одной матрицы другой является одинаковый набор чисел на соответствующих диагоналях. Оказывается, это является также и достаточным условием. Итак, допустим для матриц выполнено описанное условие, покажем тогда как из одной получить другую. Это можно сделать с помощью только ротаций квадратов 2×2 . Рассмотрим матрицу $M \times N$ и в ней произвольную диагональ, направленную вверх-направо. Допустим, перед нами стоит задача упорядочить на этой диагонали числа определенным образом так, чтобы при этом остальные элементы в матрице не поменялись. Легче всего это пояснить на примере:

		1	3	6
	15	1	2	
2	2	5		
1	3			

		1	1	6
	15	1	2	
2	3	5		
2	3			

в примере рассматривается аль с числами 1,2,1,3 (в порядке снизу вверх). Упорядочим числа на диагонали в порядке 2,3,1,1 ротации только квадратов 2×2 так, чтобы остальные числа в матрице остались на своих местах.

Заметим, что на диагонали можно поменять местами любые 2 соседних элемента, если произвести ротацию квадрата 2×2 , содержащего эти 2 элемента. К примеру, чтобы поменять порядок на диагонали с (1,2,1,3) на (2,1,1,3) надо произвести ротацию самого нижнего, левого квадрата 2×2 . Далее аналогично из (2,1,1,3) получим (2,1,3,1), а затем и (2,3,1,1). При этом так как мы использовали только квадраты 2×2 , то в матрице могли поменяться только элементы на диагонали (при каждом этапе переселения меняются только 2 народа, которые расположены на соответствующей диагонали). В общем случае если диагональ $(a_1, a_2, a_3, \dots, a_n)$ надо привести к виду $(b_1, b_2, b_3, \dots, b_n)$, то среди $a_1, a_2, \dots, a_n$ сначала находим b_1 (такой существует, так как по предположению наборы чисел $(a_1, a_2, \dots, a_n)$ и $(b_1, b_2, \dots, b_n)$ совпадают). Допустим $a_k = b_1$. Тогда преобразовываем нашу последовательность следующим образом: $(a_1, a_2, a_{k-1}, b_1, a_{k+1}, \dots, a_n) \rightarrow (a_1, a_2, \dots, a_{k-2}, b_1, a_{k-1}, a_{k+1}, \dots, a_n) \rightarrow (a_1, a_2, \dots, a_{k-3}, b_1, a_{k-2}, a_{k-1}, \dots, a_n) \rightarrow \dots \rightarrow (a_1, b_1, a_2, a_3, \dots, a_n) \rightarrow (b_1, a_1, a_2, \dots, a_{k-1}, a_{k+1}, a_n)$. Далее совершенно аналогично ставим на место $a_2 - b_2$ и так далее.

Теперь решение прозрачно. У нас есть 2 матрицы A и B одинаковых размерностей $M \times N$. По доказанному выше если существует диагональ, направленная вверх-направо, на которой набор чисел в матрицах A и B различный, то решения не существует, и в качестве ответа надо выдать -1, иначе пробегаем по всем диагоналям и упорядочиваем в матрице A элементы на этой диагонали таким образом, чтобы получилась соответствующая диагональ в матрице B. Реализовать алгоритм легче всего в 2 просмотра: сначала рассматриваем диагонали пересекающие левую сторону матрицы, затем диагонали, пересекающие нижнюю сторону. Приводим основной код на языке C++:

```
int i,j,z,d,k;
for(i = 0; i < n; i++) {
    d = min(i+1, m);
    for(j = 0; j < d; j++)
        if (his[i-j][j] != arc[i-j][j]) { //trying to find on the diagonal another element
            z = j+1;
            while (z < d && arc[i-z][z] != his[i-j][j]) z++;
            if (z == d) { printf("-1"); exit(0); }
            for(k = z; k > j; k--) {
                answer[answerNum][0] = i-k;
                answer[answerNum++][1] = k;
                swap(i-k, i-k+1, k-1);
            }
        }
}
for (i = 0; i < m-1; i++) {
    d = min(m-1-i, n);
    for (j = 0; j < d; j++)
        if (his[n-1-j][i+j+1] != arc[n-1-j][i+j+1]) { //trying to find on the diagonal...
            z = j + 1;
            while (z < d && arc[n-1-z][i+z+1] != his[n-1-j][i+j+1]) z++;
            if (z == d) { printf("-1"); exit(0); }
        }
}
```

```
        for(k = z;k>j;k--){
            answer[answerNum][0] = n-1-k;
            answer[answerNum++][1] = i+k+1;
            swap(n-1-k,i+k+1,n-k,i+k);
        }
    }
```

Задача К. Иностранные языки

Автор задачи и разбора – Б.Василевский

Рассмотрим сначала простейший случай $K = 1$. Как проверить, мог ли Вася, сказав слово T , иметь в виду P ? Для этого будем просто бежать по T одним циклом, также используя счетчик j , равный сначала 1. Как только текущий символ T совпадает с $P[j]$, надо увеличить j на единицу. Если j так и не превысил m , то не найдется последовательности, удовлетворяющей определению (иначе алгоритм бы ее нашел), и Вася не мог иметь P в виду. В противном случае --- последовательность существует и ответ положительный. Такой алгоритм работает за $O(n)$, где n --- длина T .

Если применять этот способ для каждого словарного слова, может получиться так, что по T вы пробежимся K раз --- а это слишком долго.

Для решения данной задачи используется идея, позволяющая проверять слово P длины m за m действий (конечно, с некоторой подготовкой для T).

Пусть $\text{next}[i][x]$ --- минимальная (самая правая позиция) вхождения символа x в T , начиная с $i + 1$ (здесь $0 < i < n + 1$). Другими словами,

$$\text{next}[i][x] = \min\{j: j > i \text{ и } T[j] = x\}.$$

Если такого символа нет правее позиции i , то будем считать $\text{next}[i][x] = -1$.

Чтобы вычислить next , будем бежать с конца T , поддерживая в массиве $\text{last}['a'..'z']$ искомые ближайшие позиции для каждого символа:

```
for c := 'a' to 'z' do last[c] := -1;
for i := n downto 0 do begin
  for c := 'a' to 'z' do next[i][c] := last[c];
  if (i > 0) then last[T[i]] := i;
end;
```

С помощью next легко модифицировать старый алгоритм, чтобы он уже работал за $O(m)$:

```
j := 1;
l := 0;
While (j <= m) do begin
  l := next[j][P[j]];
  If (l = -1) then break;
  j := j + 1;
End;
```

Аналогично, если j получилось больше m , то найдется последовательность, удовлетворяющая определению, то есть Вася мог иметь в виду P . В противном случае ($j <= m$) такой последовательности нет.

Чтобы проверить все K словарных слов, потребуется S (суммарная длина всех слов) + $T \cdot 26$ (расходы по вычислению next), получаем $3.6 \cdot 10^6$ в худшем случае. Как известно, такая оценка для алгоритма приемлема.

Задача Л. Переливания

Автор задачи и разбора – А.Юрьев

Докажем, что решение задачи всегда существует, а заодно построим искомую последовательность переливаний. Итак, A , B , C – количество воды в первом, втором и третьем сосудах соответственно. Пусть, для определенности, $A < B < C$ (если количество воды в каких-то двух сосудах совпадает, то решение очевидно). Далее, пусть $B = q \cdot A + r$ ($q = B \text{ div } A$, $r = B \text{ mod } A$). Рассмотрим двоичное представление числа q : пройдем в нем от младших битов к старшим, совершая для каждого разряда следующую операцию: если текущий бит q равен 0, то перельем воду из 3-го сосуда в 1-й (из C в A) – если текущий бит q равен 1, то перельем воду из 2-го сосуда в 1-й (из B в A).

Заметим, что т.к. $q \cdot A \leq B < C$, то такие переливания будут всегда возможны. Легко видно, что после такой последовательности переливаний во 2-м сосуде (B) останется как раз $r = B \text{ mod } A$ мл воды. Т. к. $r < A$, то после такой серии переливаний минимальное количество воды в сосудах уменьшится.

Продолжая действовать по изложенному выше алгоритму мы за небольшое число шагов (порядка $\log(A+B+C)$) опустошим один из сосудов. Заметим, что каждый шаг также требует от нас порядка $\log(A+B+C)$ переливаний. Таким образом, Решение всегда существует и вполне укладывается в отведенные условием ограничения (10000 переливаний для порядка $3 \cdot 10^{18}$ мл воды).